UC Redes de Computadores
ICT/UNIFESP
22/10/2026

Trabalho 1: Avaliação de Desempenho de Cliente-Servidor de Arquivos

Observações importantes:

- O trabalho deverá ser realizado por grupos de no máximo 3 alunos.
- Prazo para entrega: 05/11/2026
- Entregáveis via classroom:
  - Código(s) do servidor.
  - Video-relatório de até 10 minutos, apresente:
    - O trabalho desenvolvido (o que, como e porque).
    - Contribuições de cada integrante do grupo no desenvolvimento do trabalho.
    - Auto-avaliação do grupo sobre o trabalho realizado.
    - Suba o vídeo em sua nuvem e envie somente o link do video (**NÃO** envie o arquivo de video pelo classroom).

1) Implementação de uma Aplicação Cliente-Servidor de Arquivos

Utilizando a API de Sockets na Linguagem C, implemente uma aplicação cliente-servidor de arquivos similar. A aplicação deve atender os seguintes requisitos:

- Múltiplos clientes podem estar conectados e serem atendidos simultaneamente no servidor.

- O servidor deve executar os seguintes métodos:
  - GET *caminho_arquivo*
    Similar ao GET convencional, utilizado pelo Cliente para baixar um arquivo do Servidor. Se o arquivo existir no cliente, o mesmo deve ser substituído. Desconsidere o GET conditional.
  - PUT *caminho_arquivo*
    Similar o PUT do HTTP, o cliente envia arquivo local para o servidor. Se o arquivo existir no servidor, o mesmo deve ser substituído.

- O servidor deve responder às requisições dos clientes, conformes os métodos. Em caso de erro, o código do respectivo erro deve ser enviado como resposta ao cliente.

2) Execução da aplicação em ambiente emulado

Utilizando o emulador CORE (ver instruções de instalação no Anexo I).

2.1) Preapare o ambiente experimental:
- Crie uma topologia de rede considerando redes de acesso e de núcleo (como visto no Capítulo 1).
- Em uma das redes de acesso, crie um nó servidor para executar o servidor HTTP.
- Em outra rede de acesso, crie um nó cliente para o cliente HTTP.
- Na topologia, escolha um dos enlaces para ser o gargalo entre cliente e servidor para configurar parâmetros de vazão e latência (não configure erro).
- Configure parâmetros (de vazão e latência) de forma arbitrária e verique se os mesmos estão, de fato, consistente na emulação.
  - Para isso, execute o *ping* e o *traceroute* para verificar o RTT fim-a-fim e o RTT até cada salto (roteador), o *iperf* entre o cliente e servidor para verificar a vazão configuado no gargalo.

2.2) Defina planejamento de experimento:
- Considere os fatores:
  - Tamanhos de arquivos distintos.
    - Defina tamanhos de arquivos típicos de 2 aplicações na Internet.
  - Operações na aplicação:
    - Neste caso não apenas duas (GET e PUT).
  - Números de requisições concorrentes.
    - Defina cenários de baixa, média e alta escala de clientes.
  - Parâmetros de rede (vazão e latência) no gargalo.
    - Configure parâmetros de rede típicos de 2 tecnologias comuns de acesso à Internet.

2.3) Coleta de dados de telemetria e execução dos experimento:
- Para cada configuração experimental definida em 2.2:
  - Repita o experimento n vezes. Em cenários controlados (como no emulador), um $n > 10$ costuma ser suficiente (a variação converge entre um intervalo).
- Em cada operação (PUT e GET), colete dados de desempenho na aplicação:
  - Tempo de resposta da operação.
  - Vazão obtida na operação.

2.4) Analise os resultados:
- Após concluir os experimentos, processe e analise os resultados.
  - Sumarize os resultados com estatística básica, e.g., média, mediana, desvio padrão, intervalo de confiança.
- Escolha gráficos relevantes para apresentar os resultados, e.g., barras, linhas, CDF, boxplot.
- Identifique os gargalos de desempenho e pontos de satuação.

3) Estruture o video-relatório e grave-o.

Espera-se que o vídeo contenha ao menos:
- Introdução
  - Motivação. Problema abordado.
- Descrição do trabalho desenvolvido.
  - O que, como, porque e onde desenvolveu (considere os itens das Seções 1 e 2 deste documento).
- Discussão dos resultados obtidos.
  - Explique o desempenho observado nos cenários emulados sobre o conjunto de configurações experimentais.
- Conclusões.
  - Breve descrição do trabalho realizado e dos resultados e contribuiçõe obtidas.

ANEXO I

Instalação do CORE:

Usando o emulador da máquina disponível no laboratório:

1. Em um terminal inicie o daemon do core

```
# core-daemon
```

2. Em outro inicie o cliente

```
# core-gui
```

Instalando na própria máquina:

a. Instalação do vagrant:
   Siga a instalação de acordo com seu sistema no site oficial do vagrant
b. Inicialização da VM
   Crie uma arquivo de configuração de VM padrão:

```
# vagrant init hashicorp/bionic64
```

Inicialize a VM (pode demorar um pouco) e reinicie a VM:

```
# vagrant up && vagrant reload
```

Reinicie e acesse a VM:

```
# vagrant ssh
```

c. Instalação docker
   Instale os pacotes necessários para o docker:

```
# sudo apt install apt-transport-https ca-certificates curl
software-properties-common
```

Adicione a chave GPG oficial do docker:

```
# curl -fsSL https://download.docker.com/linux/ubuntu/gpg |
sudo apt-key add -
```

Adicione docker ao repositório apt:

```
# sudo add-apt-repository "deb [arch=amd64]
https://download.docker.com/linux/ubuntu bionic stable"
```

Faça update do apt:

```
# sudo apt update
```

Instale docker:

```
sudo apt install docker-ce docker-ce-cli containerd.io
```

d. Instalação do emulador de reder CORE

Seguindo os passos da instalação do site oficial, clone o repositório git e vá para sua pasta:

```
# git clone https://github.com/coreemu/core.git && cd core
```

Faça o build do EMANE python:

```
# sudo docker build -t emane-python -f
dockerfiles/Dockerfile.emane-python .
```

Faça o build da imagem do core

```
# sudo docker build -t core -f dockerfiles/<Dockerfile> .
```

* Substitua <Dockerfile> por Dockerfile.rocky ou Dockerfile.ubuntu

Inicie o container:

```
# sudo docker run -itd --name core -e DISPLAY -v /tmp/.X11-
unix:/tmp/.X11-unix:rw \
    --privileged --init --entrypoint /opt/core/venv/bin/core-
daemon core
```

Habilite a gui para core:

```
# xhost +local:root
```

Execute core no container criado:

```
# sudo docker exec -it core core-gui
```